Analyzing Password Bias with BERT and Graphical Modeling

Siddhi Kabadi & Ena Nayak
Santa Clara University

***Abstract*—Password bias occurs when users create passwords with keywords containing personal information, current events, cultural references, or easily memorized sequences for the sake of convenience. Regarding security, this poses a major threat as such passwords are vulnerable to brute force and dictionary attacks. Researchers often describe password strength as a combination of four main features: overall length, usage of uppercase characters, frequency of symbols, and consistency of digits. We utilize machine learning frameworks to study these features in the context of password strength and analyze bias. We have fine-tuned a pre-trained BERT model (bert-base-cased) on a labeled password dataset, which classifies passwords as easy, neutral, or difficult (quantified numerically as 0-2). We gather insights by testing the model on a database of unclassified passwords, storing its subsequent classifications, performing graphical modeling with the outputs, and applying KMeans clustering to identify patterns within each grouping. Lastly, we visually compared the model's classifications to those of the 2016 published Z library for password classification. Using scikit-learn, we created a Correlation Matrix to compare the results of our BERT model to the Z library. To further study the correlation between the four main features and password strength, we developed a heatmap using Python's matplotlib. These visualizations showcased a strong level of agreement between both password strength classifiers. Across the results of both models, we saw that symbol usage and password length are statistically significant to highly ranked passwords.**

I. Introduction

Password bias is a phenomenon where users create passwords using keywords or dates containing references to personal information, current events, cultural references, or easily memorized sequences. Security-wise, many users select passwords that are guessable or easy to brute force, and are therefore subject to dictionary attacks. Vulnerable password practices often lead to violations in confidentiality, integrity, and availability. Researchers believe that susceptibilities in passwords are psychological in nature. In a 2015 study published by the National Institutes of Health, researchers investigated the psychological factors behind password bias. They concluded that impulsivity, a locus of control, familiarity bias, and a lack of cybersecurity knowledge contribute to weak password defenses [12].

In 2018, the City of Atlanta faced a ransomware attack where attackers brute-forced their way into networks connected to Atlanta's City Hall. The attackers leveraged algorithmic password-cracking techniques to access confidential credentials. Upon obtaining the credentials, they launched an attack using SamSam ransomware, a customized malicious software. Once these networks were compromised, the attackers demanded $50,000 in Bitcoin, compromising critical technologies. Key municipal functions within several city departments were disrupted, such as online payment programs for utilities, traffic tickets, and business licensing. Additionally, many law enforcement operations were disrupted, including warrant issuances, inmate processing protocols, and court fee payments. Atlanta's government refused to pay the attackers, forcing the city to recover independently. This attack emphasizes the importance of password uniqueness, multi factor authentication, and the enforcement of least privilege [6].

We highlight the three following peer-reviewed articles as motivation and contributors to the direction of our study.

1. **zxcvbn: Low-Budget Password Strength Estimation (2016, USENIX Security Symposium):** In this paper, researchers analyze the weaknesses of popular password feedback mechanisms. Researchers compare zxcvbn, an open-source password classifier, to LUDS (lowercase, uppercase letters, digits, and symbols), a traditional password strength estimator. Their work advocates for zxcvbn, and classifies LUDS as an outdated mechanism that is unfeasible and ineffective. Their conclusions indicate how zxcvbn is an enhanced, user-friendly, and more secure implementation of LUDS. [1]
2. **"Guess Again (and Again and Again): Measuring Password Strength by Simulating Password-Cracking Algorithms" (2021, IEEE):** Researchers analyzed 12,000 passwords collected under seven composition policies via an online study and developed an efficient distributed method for calculating how effectively several heuristic password-guessing algorithms guess passwords. This method was leveraged to investigate password resistance under different conditions, the

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

performance of guessing algorithms with different training sets, the relationship between passwords created under a certain policy and passwords that inherently meet the same requirements, and the relationship between guessability and entropy estimates [3].

3. **"Reducing Bias in Modeling Real-world Password Strength via Deep Learning and Dynamic Dictionaries" (Usenix, 2021):** Researchers used deep neural networks and dynamic guessing strategies to model an adversary's attack building configurations and introduce dynamic guessing strategies within dictionary attacks, mimicking experts' ability to quickly adapt their guessing strategies by incorporating their knowledge of their targets [2].

Goal & Method

Our primary goal is to investigate the factors that constitute a strong password. To do this, we trained a BERT model on a classified dataset of passwords and tested the model on two unclassified sets of passwords. Using our model's classifications of the test data, we created visualizations of common attributes and patterns of model's classifications. We then compared our model's outputs and noticeable patterns to those of the z library. We checked for patterns on four key features: length, uppercase letters, digits, and symbols. As these features are also considered within the LUDS classification method, studying these trends in the context of BERT helps us conclude whether LUDS is truly an outdated mechanism.

BERT

BERT (Bidirectional Encoder Representations from Transformers) is a 2018 language model. It utilizes Transformer architecture to analyze textual relationships bidirectionally, considering both left-to-right and right-to-left contexts. We opted for BERT in this project due to its fine-tuning capabilities, ability to classify texts, and its proficiency in using deep learning to study relationships between tokens.

II. Implementation

A. Data Preparation

Our training dataset was obtained from Kaggle [8], and consisted of two different columns: password and strength. The strength column contains values 0, 1, and 2, which represent easy, medium, and hard, respectively. To prepare the data, we had to delete rows with NaN values. Because the labels were already represented as integer values, we did not need to encode the labels as integers. Once the data was cleaned, we used the pandas library to convert the dataset into a dataframe. Due to computational restraints, we took a sample of 70% of the dataframe, still ensuring that the distribution of the labels remained the same. The model was trained with 70% of the sampled data, where 30% was reserved for testing. Our training dataset had a total of 468,747 entries, which are classified as weak, medium, or strong, or 0, 1, or 2, respectively. The dataset consisted of 62,791 weak examples, 347,760 medium examples, and 58,196 strong examples.

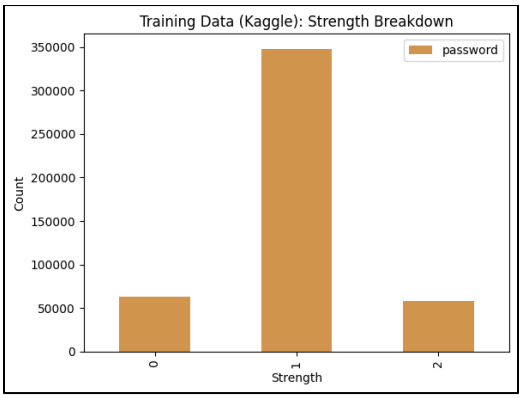

Figure 1: *Categorization of the Kaggle data set and how the passwords were broken down*

B. Importing the Bert

We loaded the model bert-base-cased, which was imported from the python transformers library. We specifically selected bert-base-cased because bert-base, specifically, is a smaller bert model [5]. This is helpful to avoid possible memory restraints. Additionally, the model is trained to differentiate between uppercase and lowercase tokens [9]. Regarding password security, we were interested in the effect of letter casing on password strength. Thus, we needed a model that was trained to differentiate between the cases. We added a linear classifier to the model from pytorch, implementing this such that the BERT model and the linear classifier are combined into one PyTorch neural network. Lastly, we moved the model to the GPU.

C. Tokenizing the Data

A tensor is a data structure, similar to a matrix, used to encode the inputs and outputs of a model in a mathematical format [10]. As a result, the model can perform the necessary computations using the inputs. Knowing this, we converted the strength column into a PyTorch tensor and encoded the passwords column. The tokenizer was imported from the transformers library and initialized to bert-base-cased. We initialized a maximum length to 512, where the inputs would either be padded or truncated to this length. Lastly, the return tensors were specified as PyTorch tensors, since we were using tools from the torch library. While encoding the password column, we saved the input IDs (the actual mathematical representations of the text) and the attention masks (differentiates between the actual input and the padding). These generated input IDs, attention masks, and strength column tensors were then saved as TensorDataset objects, imported from the library torch.utils.objects. As discussed in the previous sections, our training data was split into 70% training and 30% testing. Encoding the strength column in this manner was not necessary, since this process was completed for both the training and testing datasets.

D. Training & Evaluation

We trained bert-base-cased for three iterations, or epochs through the training data. We batched the data by loading the TensorDataset objects for training and testing as Dataloader objects, imported from the torch.utils.data library, so that the model could process multiple inputs at the same time. To avoid memory constraints, we selected a smaller batch size of eight. The learning rate for training was set to 2e(-5) using the

AdamW optimizer from the torch.optim library to update model parameters each iteration. To properly train the model, it is necessary to track the model's loss, which is essentially the difference between the model's predictions and the true labels. To get this value, we obtained outputs by inputting the input IDs and attention masks through bert-base-cased, which had to be moved to the GPU prior. From there, we extract pooled outputs, the summary of the sequence and get the logits, the model's predictions, by running the pooled outputs through a linear classifier, imported from the torch library. Then, we use cross entropy loss to get the total loss by comparing the logits to the true labels. Based on the loss, the model updates the model's parameters and continues onto the next iteration.

After training for three iterations, we needed to evaluate the accuracy of the model by testing using the reserved test samples. We generated the accuracy score and the F1 score. The accuracy score alone may not be the best indicator of the model's performance. For instance, if our model could predict one label class accurately, while failing to predict the other classes correctly, this score could still be shown as high. Thus, using the F1 score, we observe how well the model predicts the true labels, while minimizing the occurrence of false labels. Additionally, the F1 score can be a good indicator of whether the model is learning dependencies and patterns, or simply memorizing the training data. Generally speaking, an accuracy score closer to 100% and an F1 score closer to 1 indicate that the model is performing well. To evaluate the model, we used a similar method to the model training. There were some differences; we did not calculate gradients or update any parameters. Using the loss, we could simply calculate the accuracy and F1 score using the respective functions, imported from sklearn.metrics. Our model received an accuracy of approximately 87% and an F1 score of 0.81, which indicates that our trained model was able to correctly identify patterns and classify passwords on the testing dataset. This trained model was then saved for testing.

```
Training Loss: 0.3506726136717016
Arruracy: 0.8690702222222222
F1 Score: 0.8133530671739548
```

Figure 2: *Model's Statistics Output*

E. Testing BERT on Unclassified Passwords

To evaluate the performance of our BERT model, we conducted tests on two different datasets. The first consisted of 10,000 common passwords [13] and the second consisted of 10,000 randomly generated passwords [14], where password strings were randomly generated using combinations of letters (uppercase and lowercase), digits, and symbols of arbitrary lengths [14]. These datasets included extra information, such as the count of letters, symbols, and digits used; however, our interests only lied in the password strings. For both datasets, we encoded the passwords in the same method used for the password column in the training and testing datasets. Similar to our evaluation step, we passed in these inputs to the model and extracted the outputs and logits. We converted the logits to the actual prediction scores (0, 1, and 2), and created two new data frames with the passwords and their BERT classifications.

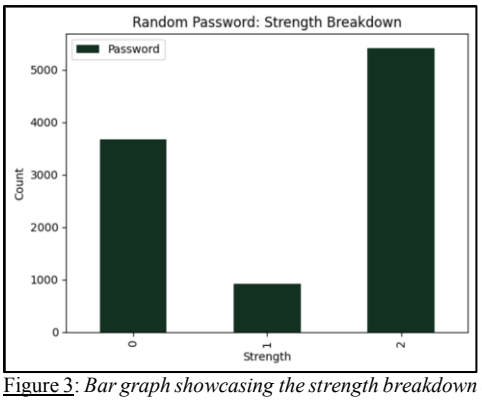

Figure 3: *Bar graph showcasing the strength breakdown of randomly generated passwords*

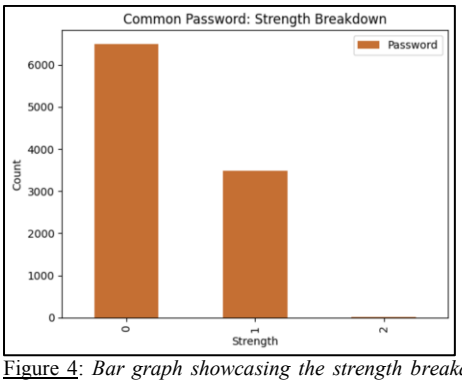

Figure 4: *Bar graph showcasing the strength breakdown of commonly used passwords*

III. Generating Graphical Insights

To further understand the relationship between the characteristics of a password and its strength, we generated a series of graphical insights using Python libraries. For the BERT outputs on randomly generated and commonly used passwords, we created bar charts to visualize the average values of the four key features: password length, symbol count, digit count, uppercase letter usage, and their relationship to the different strength classifications. We also created correlation heatmaps to explore the interdependencies between each feature, and their impact towards strength classifications.

A. Bar Charts and Heat Maps

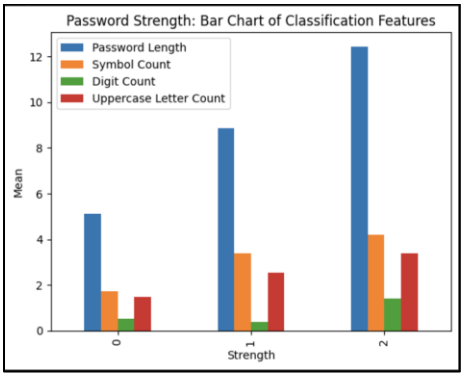

Figure 5: *Bar chart of password strength features on randomly generated passwords*

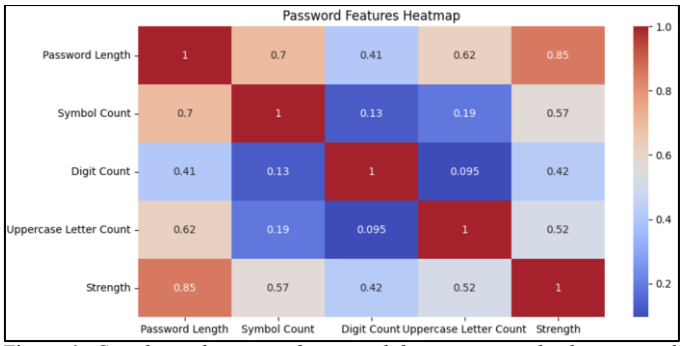

Figure 6: *Correlation heatmap of password features on randomly generated passwords*

The graphical analysis of BERT's output on randomly generated passwords demonstrates how passwords with a strong classification ranking tend to have higher average values in length and symbol usage. The correlation heatmap provides the same conclusion, as there is a 0.85 correlation value bertween password length and strength, and a moderate value of 0.57 between strength and symbol usage. The bar chart and heatmap serve as empirical evidence that passwords possessing symbols and larger overall lengths are classified at higher strength levels.

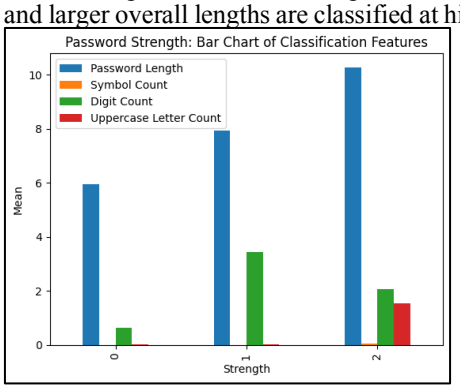

Figure 7: *Bar chart of password strength features on commonly used passwords*

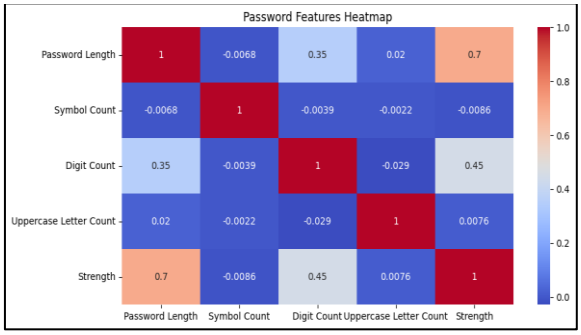

Figure 8: *Correlation heatmap of password features on commonly used passwords*

Figure 7 showcases a similar result, as password length and symbol usage increase along with the classifications levels. The heatmap reinforces these observations as password length has the highest correlation value, 0.7, to strength levels. This is followed by digit count, as it has a value of 0.45. However, it is important to note that this dataset contains an inherent bias. Specifically, it is skewed towards weak rankings, as the passwords lack complexity and diversity in features.

B. KMeans Clustering

KMeans clustering is an unsupervised machine learning method that clusters data based on their similarities. Ideally, data within the same cluster has a strong agreement level, while data in varying clusters have different characteristics [14]. We were particularly interested in analyzing the specific patterns in the BERT classifications. In order to gain further insight into the classifications of our BERT model, which ranks passwords on a scale of 0, 1, and 2, we used Python's Pandas library to categorize the random passwords into CSV's based on their ranking, leveraging KMeans to identify the underlying patterns between each classifcation.

We specifically studied the clustering between passwords and the number of digits and uppercase letters that they possess.

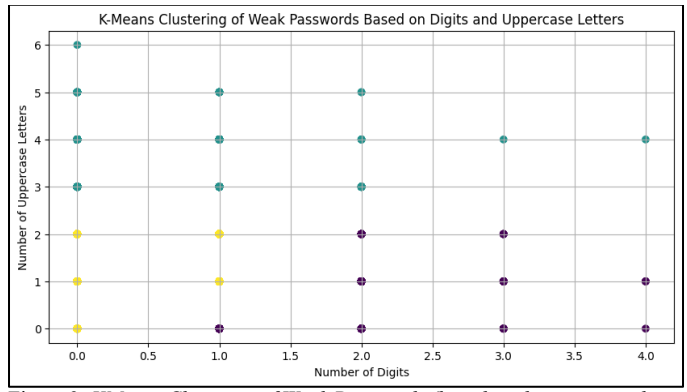

Figure 9: *KMeans Clustering of Weak Passwords (based on digit count and uppercase letter usage)*

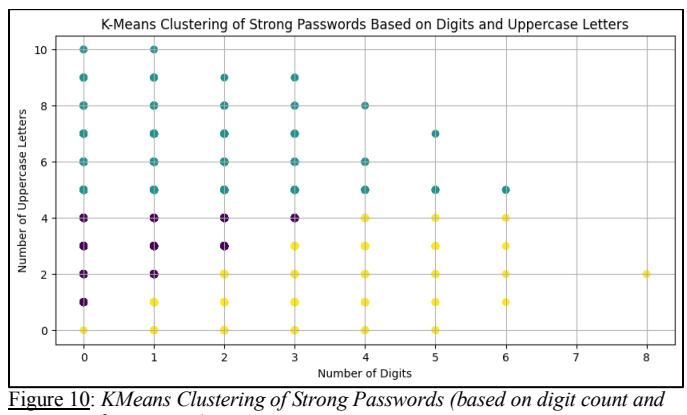

Figure 10: *KMeans Clustering of Strong Passwords (based on digit count and uppercase letter usage)*

In Figure 9 and 10, three different clusters are identified by the KMeans algorithm. Cluster 1 encompasses passwords with minimal complexity, as there is a scarcity in the number of digits and uppercase letters used. Cluster 2 includes passwords of average strength. Passwords within this cluster tend to have a few more digits and uppercase characters, but not enough to be classified as strong. The last group, Cluster 3, contains passwords with a significant amount of digits and uppercase letters. There is also a noticeable difference in the number of points that each graph projects. Figure 9 contains fewer points than Figure 10, as there is limited diversity in the kind of features used in weak passwords. It can be concluded that weak passwords are homogenous in nature, as they lack complexity and variability.

IV. BERT VS ZXCVBN

In the article, Low-Budget Password Strength Estimation, the efficiency of LUDS (lowercase, uppercase, digits, and symbols) as a password strength evaluator is studied. Researchers explain how simply meeting the requirements of LUDS is not enough to constitute a secure password. They argue that LUDS is difficult to implement as it violates the security principle of considering human factor. A strict requirement on the kind of characters used in a password is not feasible. Researchers instead argue that entropy, a measure of randomness, is a more secure way to rank passwords. They advocate for the usage of zxcvbn, a open-source library, that analyzes patterns within passwords. Rather than focusing on the amount of characters utilized in a password, zxcvbn studies patterns, such as character repetition, keyboard proximity, and predictable substitions (such as l33t speak).

To understand the different approaches in password security, we compared how BERT classifies randomly generated passwords to the Z library.

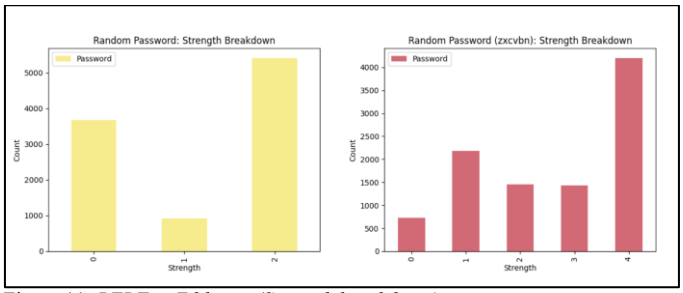

Figure 11: *BERT vs Z library (Strength breakdown)*

The Z library ranks passwords on a different numerical scale than BERT. It classifies passwords as 0: weak, 1: somewhat weak, 2: moderate, 3: moderately strong, 4: strong. To accurately compare the results between BERT and the Z library, we established equivalences between both scales. A BERT score of 0 was mapped to Z library scores of 0 and 1. A BERT score of 1 was set to the Z library score of 2, and a BERT score of 2 was mapped to the Z library score of 3. When applying this scale to Figure 13, it is clear that both models classify passwords in a similar manner.

Using the Scikit-learn library, we quantified the differences in classifications made by each model. We created a Confusion Matrix to numerically evaluate the alignment between both systems, and identify the areas in which classifications diverge.

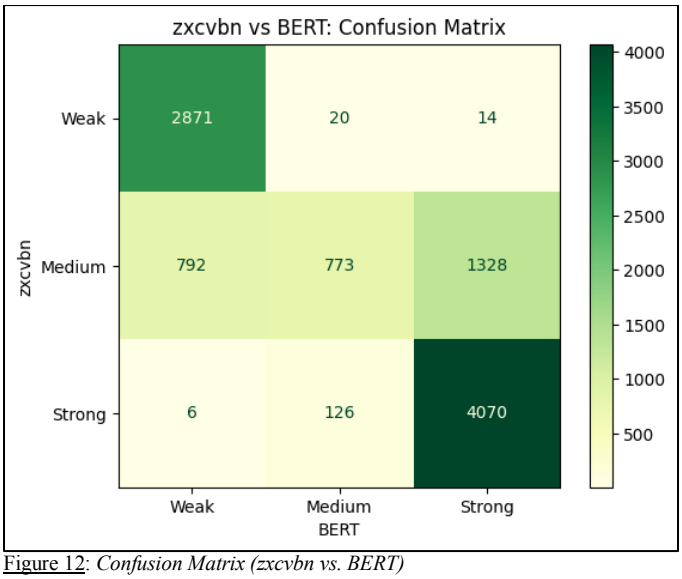

Figure 12: *Confusion Matrix (zxcvbn vs. BERT)*

Figure 12 conveys a general consensus between the classifications of the BERT model and Z library. The models both ranked 2871 passwords as weak, 773 passwords as medium, and 4070 passwords as strong, indicating that there is general agreement between how the BERT model and Z library determine password strength. We can see that there is a slight variance in how the models classify medium passwords, but this can be due to differences in the numerical scale, and the manner in which the Z library classifies passwords. The exact methodology of how the Z library classifies passwords is not publicly documented, which limits our ability to accurately compare it to BERT and LUDS.

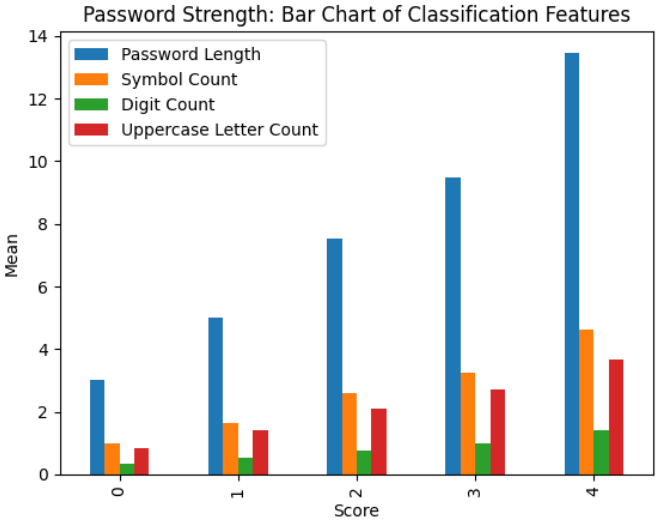

Figure 13: Z library: Feature and Strength Bar Chart

Figure 13 presents a bar chart on the average values of the four password features to each classification level. Similar to Figure 5, a bar chart of BERT's output, the usage of all features consistently increases with the score. Both the Z library and BERT model consider password length, symbol count, digit count, and uppercase letter count when classifying passwords.

V. Future Considerations & Conclusion

There are future considerations we aim to investigate, specifically regarding the selection and the training of the language model. For instance, BERT is a predecessor of RoBERTa (a Robustly Optimized BERT Pretraining Approach) [15]. It may beneficial to analyze this model's insights after fine-tuning and compare the results to those of BERT and the Z library. Additionally, we consider the parameters used specifically during BERT's training, including the number of iterations spent training, the learning rate, and the batch sizing. It may be possible that increasing the number of iterations during training may be better for BERT to better learn dependencies in patterns. Additionally, different learning rates and batch sizes may be yield more optimal results in terms of accuracy, which can only be determined through further testing.

All of our studies on BERT show that there is a correlation between the features of length, uppercase characters, symbols, and digits, to password strength. These four features are also classified within LUDS, which suggests that there may be a connection between BERT's classification and the theory behind LUDS. As there is also an agreement between BERT and the Z library, we can deduce that the Z library still draws upon some of the principles of LUDS. Even though researchers criticize LUDS and deem it as unreliable and inefficient, many of its characteristics still exist within the zxcvbn library.

References

[1] D. L. Wheeler and D. Lowe, "Open access to the Proceedings of the 25th USENIX Security Symposium is sponsored by USENIX zxcvbn: Low-Budget Password Strength Estimation," in *Proceedings of the 25th USENIX Security Symposium*, 2016. [Online]. Available: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_wheeler.pdf

[2] D. Pasquini, M. Cianfriglia, G. Ateniese, and M. Bernaschi, "Reducing Bias in Modeling Real-world Password Strength via Deep Learning and Dynamic Dictionaries," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021. [Online]. Available: https://www.usenix.org/system/files/sec21-pasquini.pdf

[3] P. G. Kelley et al., "Guess Again (and Again and Again): Measuring Password Strength by Simulating Password-Cracking Algorithms," *IEEE Transactions on Dependable and Secure Computing*, May 2012. [Online]. Available: https://ieeexplore.ieee.org/document/6234434

[4] Technocrat, "Fine-tuning BERT for Text Classification: A Step-by-Step Guide," *Medium*, Sep. 13, 2023. [Online]. Available: https://medium.com/@coderhack.com/fine-tuning-bert-for-text-classification-a-step-by-step-guide-1a1c5f8e8ae1

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," May 2019. [Online]. Available: https://arxiv.org/pdf/1810.04805

[6] Coverlink, "City of Atlanta Ransomware Case Study," 2018. [Online]. Available: https://coverlink.com/case-study/city-of-atlanta-ransomware/

[7] D. Pasquini et al., "Presentation: Reducing Bias in Modeling Real-world Password Strength via Deep Learning and Dynamic Dictionaries," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/pasquini

[8]Kaggle, "Password Strength Classifier Dataset," [Online]. Available: https://www.kaggle.com/datasets/bhavikbb/password-strength-classifier-dataset

[9] Google, "BERT Base Cased Model," Hugging Face. [Online]. Available: https://huggingface.co/google-bert/bert-base-cased

[10] PyTorch, "Tensor Documentation," [Online]. Available: https://pytorch.org/docs/stable/tensors.html

[11] NIH, "A Guide to Understanding Systematic Reviews," *National Institutes of Health*, [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC4291202/

[12] Kaggle, "10000 Common Passwords Dataset," [Online]. Available: https://www.kaggle.com/code/shivamb/10000-common-passwords

[13] Kaggle, "Random Passwords Dataset," [Online]. Available: https://www.kaggle.com/datasets/jeffersonvalandro/password-dataset

[14] IBM, "K-means Clustering," [Online]. Available: https://www.ibm.com/topics/k-means-clustering.

[15] L. Zhang, J. Stokes, and R. Wang, "Cost-aware Active Learning for Intrusion Detection," *arXiv preprint arXiv:1907.11692*, 2019. [Online]. Available: https://arxiv.org/pdf/1907.11692.